实验一：智能合约基础与以太坊机制初探

1. 实验目标

1. **掌握开发环境**

熟悉 Remix IDE（https://remix.ethereum.org/），这是一个用于编写、编译和部署智能合约的基于浏览器的集成开发环境。

2. **掌握 *Solidity* 基础**

学习 Solidity 合约的基本结构，包括代码版权声明（License）、编译器版本（Pragma）和合约定义（Contract）。

3. **理解以太坊账户模型**

探索以太坊中的两类账户：

- 外部账户 (Externally Owned Accounts, EOA)
- 合约账户 (Contract Accounts)

4. **探索区块链操作的经济模型**

理解以下两者之间的区别：

- 改变状态的交易（消耗 Gas）
- 只读调用（在本地执行，不消耗 Gas）

2. 实验背景：去中心化捐款箱

传统的捐款箱往往透明度较低。收集到的资金可能被管理员挪用，且捐赠者无法验证资金的具体去向。 区块链技术提供了一个透明且可验证的账本。在本实验中，你将实现一个去中心化的捐赠合约（Tip Jar），确保：

- **透明性** – 任何人都可以检查区块链上的合约余额和捐赠记录。
- **通过代码逻辑实现安全性** – 资金只能根据智能合约中预定义的规则进行处理。 本实验展示了如何通过程序化规则在去中心化系统中建立信任。

3. 实验环境

开发工具：Remix IDE（在线版，访问地址：https://remix.ethereum.org/）

开发语言：Solidity

编译器版本：0.8.20

测试环境：Remix VM

4. 实验原理与核心概念

（一）以太坊账户模型

外部账户（EOA）：即 Remix 操作面板中 ACCOUNT 下拉菜单显示的地址，由私钥进行管控，是用户开展交易、与智能合约交互的核心载体，可主动发起各类交易操作。

合约账户：智能合约部署后，在 Deployed Contracts 区域生成的专属地址，无对应私钥，完全由合约代码逻辑控制，会根据外部触发的操作自动执行预设的代码指令，属于可编程的自治系统。

（二）Solidity 核心语法与内置元素

pragma：用于指定 Solidity 编译器的版本，保障合约编译过程的版本兼容性，避免因版本不匹配出现编译错误。

msg.sender：获取当前交易或调用的发起者地址，是智能合约中实现权限控制、身份识别的核心内置变量。
msg.value：获取随交易一同发送的 ETH 数量，计量单位为 Wei（1 ETH = 10^18 Wei），该变量仅在被 payable 修饰的函数中有效。
address (this).balance：获取当前合约账户中存储的 ETH 总余额，是查询合约资金状态的核心语句。
require ()：条件校验核心函数，若校验条件结果为 false，会立即回滚整个交易流程，同时释放未被消耗的 Gas，是保障合约安全的关键方法。
block.timestamp：获取当前区块生成的时间戳，计量单位为秒，可用于在合约中实现时间相关的限制逻辑。
mapping：Solidity 中的键值对映射数据结构，适用于快速实现身份标记、状态存储等功能，如黑名单管理、捐赠者状态标记等场景。
函数修饰符：payable 表示该函数可接收 ETH 转账；view 表示该函数为只读调用，不会修改合约的状态数据，执行过程也不消耗 Gas。
（三）区块链交易经济模型
改变状态的交易：指会对区块链上的合约状态产生修改的操作（如捐赠转账、拉黑地址、提取合约资金等），这类操作需要全网节点验证并记录在区块中，因此会消耗 Gas，交易发起者需支付相应的 Gas 费用（测试环境中为虚拟 Gas，无实际价值）。
只读调用：指仅对合约状态进行查询的操作（如查询合约余额、管理员地址、黑名单列表等），这类操作可在本地节点独立执行，无需全网节点达成共识，因此不会消耗 Gas。

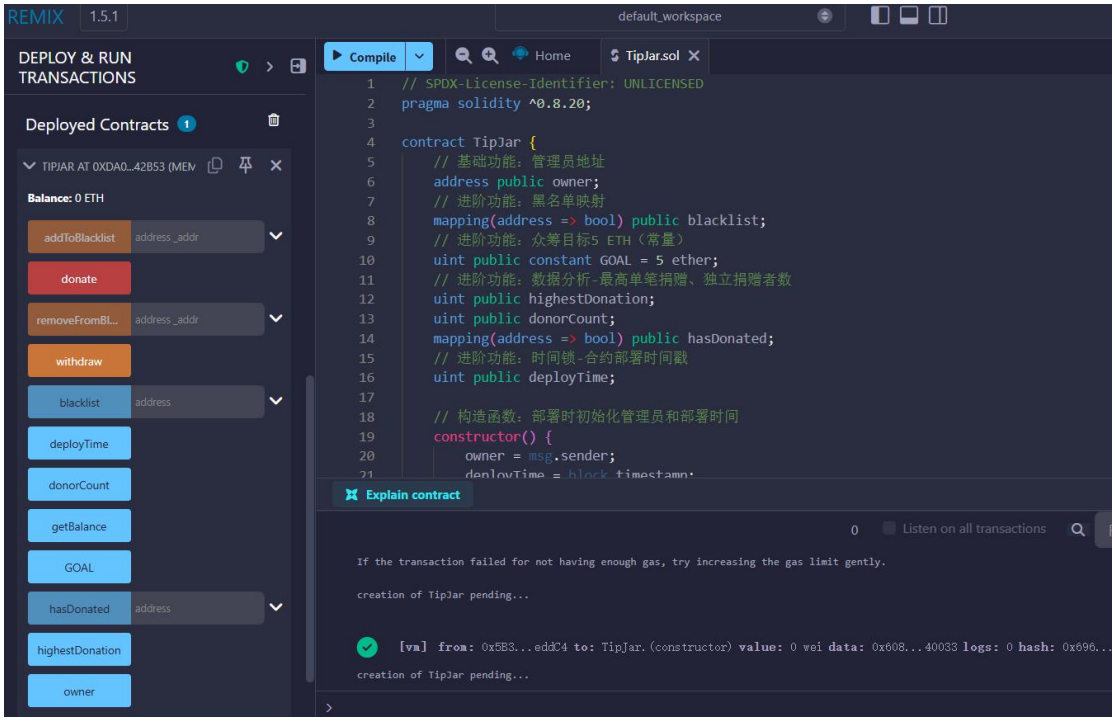
5. 实验内容与步骤
（一）编写 TipJar 智能合约
在 Remix IDE 左侧的文件管理面板中，点击「Create new file」按钮，新建一个名为 TipJar.sol 的合约文件。
按照实验要求，编写包含基础功能与进阶功能的完整 Solidity 代码，实现管理员机制、捐赠接口、余额查询、资金提取等基础功能，同时开发黑名单、众筹目标、数据分析、时间锁等进阶功能，确保代码无语法错误和逻辑漏洞。
（二）编译智能合约
点击 Remix IDE 左侧的「Solidity 编译器」面板，在编译器版本的下拉选择框中选定 0.8.20，与合约代码中指定的编译器版本保持一致。
可开启「Auto compile」自动编译功能，同时点击「Compile TipJar.sol」进行手动编译，直至面板出现绿色对勾标识，代表合约编译成功，无任何语法错误和警告提示。
（三）部署智能合约
点击 Remix IDE 左侧的「Deploy & Run Transactions」面板，在「Environment」下拉选择框中选定 Remix VM（实验指定的测试环境）。
确认当前选中的 ACCOUNT 为系统自动分配的测试账户（带有测试 ETH），在「Contract」下拉选择框中选定「TipJar - TipJar.sol」，确保待部署的为目标合约。
点击橙色的「Deploy」按钮，等待合约部署完成。部署成功后，下方 Terminal 面板会显示该次部署的交易哈希，右侧「Deployed Contracts」区域会生成 TipJar 合约实例，同时展示该合约的专属地址。
（四）合约功能交互与测试
合约部署成功后，利用 Remix VM 提供的多个测试账户，切换不同的 ACCOUNT 地址，对合约的所有功能进行全面的测试验证，观察 Terminal 面板的交易日志与合约状态的变化情

况，验证各项功能是否符合实验要求，核心测试步骤如下：
合约基础信息初始化验证
捐赠功能执行与合约余额增加测试
黑名单机制生效性测试
数据分析功能准确性测试
资金提取操作的权限与时间锁机制测试

6. 实验结果与分析

（一）部署结果

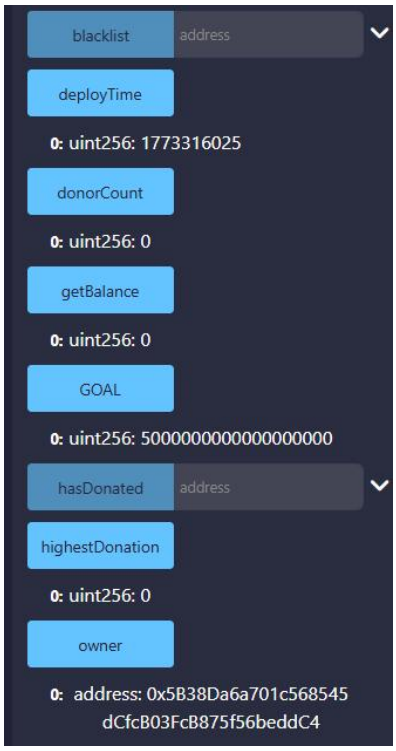
本次实验成功将 TipJar 智能合约部署至 Remix VM 测试环境，部署完成后，在 Deployed Contracts 面板可正常显示合约实例及其唯一的合约地址，Terminal 面板无任何报错信息，合约的初始化参数（owner、deployTime、GOAL）均与预设值一致，为后续的合约功能测试奠定了坚实的基础。



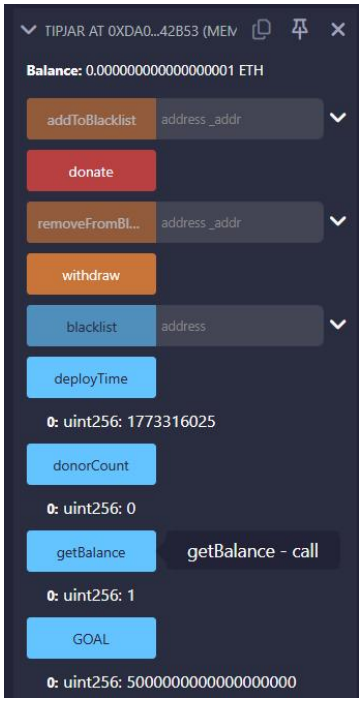
（二）功能验证结果

本次实验对 TipJar 合约的基础功能和进阶功能开展了全面的测试验证，所有功能均达到实验预设要求，具体验证结果如下：

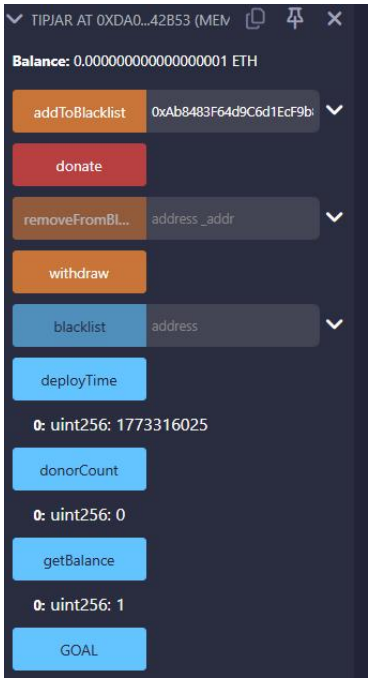
基础信息验证：点击合约实例中的 owner、getBalance ()、GOAL、deployTime 等只读函数，验证结果显示管理员地址初始化正确、合约初始余额为 0、众筹目标与部署时间戳均符合预设，基础信息初始化无异常。

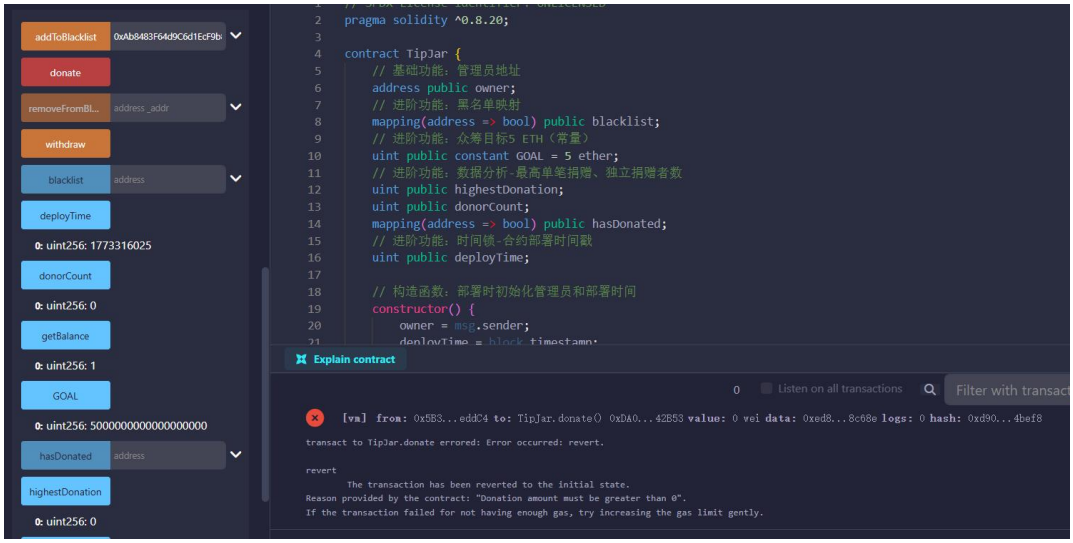


捐赠功能与余额增加测试：切换至非管理员的普通测试账户，在 VALUE 输入框中填写捐赠的 ETH 数量（如 1 ETH），点击 donate () 橙色按钮发起捐赠交易，交易成功后再次调用 getBalance () 函数，验证结果显示合约余额随捐赠操作同步增加，捐赠功能正常生效。

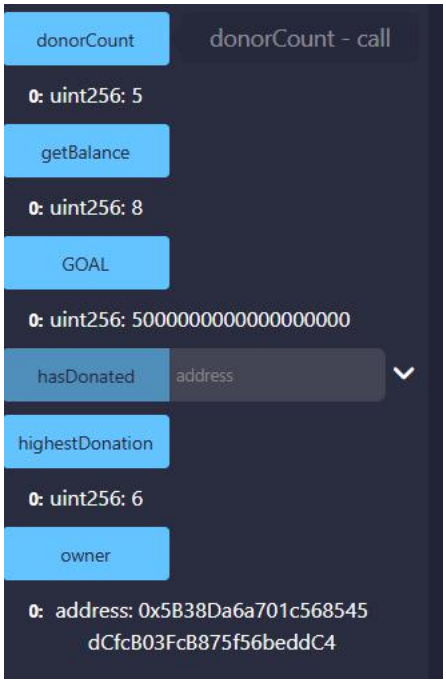


黑名单机制测试：切换回管理员账户，在 addToBlacklist () 函数的输入框中填写某一普通测试账户的地址，点击执行将该地址加入黑名单；随后切换至该被拉黑的账户，尝试发起捐赠操作，Terminal 面板出现交易失败提示（Address is blacklisted），验证黑名单机制可有效拦截被拉黑地址的捐赠行为。

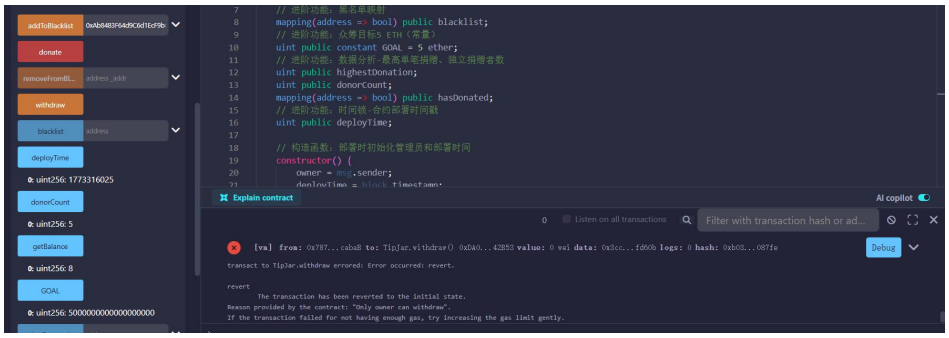




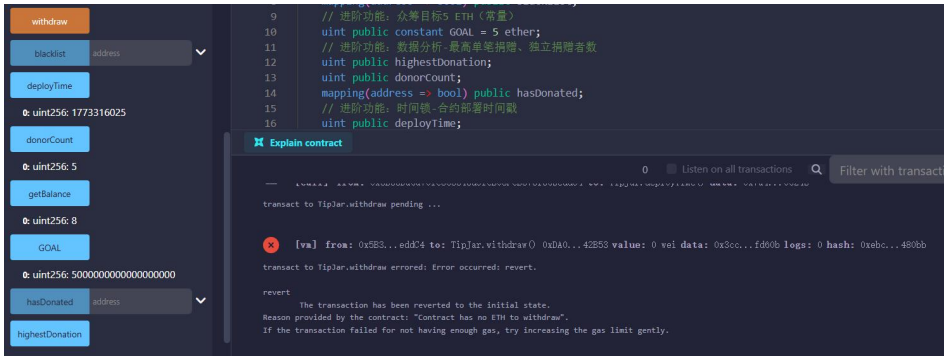
数据分析功能测试：从多个不同的测试账户发起多次捐赠操作，验证结果显示 highestDonation 会自动更新为所有捐赠中的单笔最大金额；使用同一账户发起多次捐赠，验证结果显示 donorCount（独立捐赠者数）仅进行一次计数，hasDonated 映射可准确标记捐赠者的捐赠状态，数据分析功能准确有效。



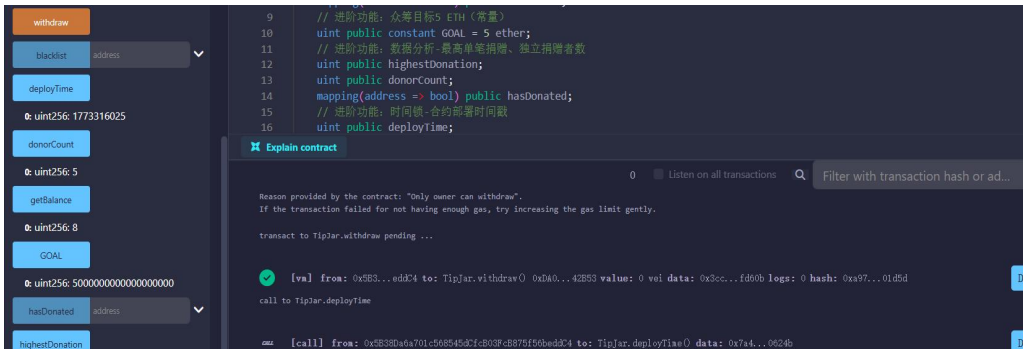
权限控制测试：切换至非管理员的普通测试账户，调用 withdraw () 资金提取函数，验证结果显示交易失败并出现「Only owner can withdraw」的提示，说明合约的管理员权限控制机制可有效拦截非管理员的资金提取操作。



时间锁冷却期测试：合约部署后的 5 分钟内，切换至管理员账户，调用 withdraw () 资金提取函数，验证结果显示交易失败并出现时间锁相关错误提示，说明合约的时间锁冷却期机制可有效限制部署初期的资金提取操作。



资金提取功能测试：等待合约部署满 5 分钟后，使用管理员账户再次调用 withdraw () 函数，验证结果显示交易成功，合约余额清零，说明资金提取功能在满足权限和时间锁条件后可正常执行。



（三）核心机制验证结果

以太坊账户模型验证：通过切换不同的 EOA 账户发起交易、调用合约，观察到合约账户可根据外部操作自动执行预设逻辑，清晰验证了 EOA 由私钥管控、可主动发起交易，合约账户由代码逻辑控制、被动执行操作的核心特征。

区块链经济模型验证：所有修改合约状态的操作（捐赠、拉黑地址、提取资金）均会触发交易并消耗 Gas，而仅进行状态查询的只读操作（查余额、查管理员）无任何 Gas 消耗，验证了状态交易与只读调用的本质差异。

Solidity 语法有效性验证：require () 条件校验函数可有效拦截各类非法操作并回滚交易，mapping 数据结构能高效实现身份标记功能，block.timestamp 可精准实现时间限制逻辑，证明本次实验所使用的 Solidity 核心语法和内置元素具备良好的实用性和有效性。

（四）实验问题与解决

本次实验过程中，在合约部署阶段出现了 invalid opcode 的报错问题。经排查，该问题的成因是 Solidity 编译器的 EVM 版本与 Remix VM 运行环境的 EVM 版本不兼容；对应的解决方法为在编译器的高级配置界面中，将 EVM 版本调整为与 Remix VM 一致的版本，重新编译合约后再次部署，最终部署成功。该问题也验证了在智能合约开发过程中，开发环境配置一致性的重要性。

7.实验总结与思考

（一）实验总结

本次实验通过开发并实现去中心化的 TipJar 捐赠合约，完成了智能合约从开发、编译、部署到功能测试的全流程操作，不仅熟练掌握了 Remix IDE 的各类操作方法，还深入理解了 Solidity 合约的基础结构与核心语法规则。

通过本次实验，精准区分了以太坊外部账户与合约账户的核心特性，明确了区块链中状态交易与只读调用的 Gas 消耗机制，成功实现了管理员控制、黑名单、众筹目标、时间锁等核心功能，验证了区块链技术在提升捐赠流程透明度、保障资金安全方面的技术优势，也深刻理解了程序化规则在去中心化系统中建立信任体系的核心逻辑。

同时，在实验过程中成功解决了环境版本兼容的问题，积累了智能合约开发与调试的基础经验，为后续开发更复杂的智能合约奠定了良好的基础。

（二）实验思考

合约安全优化：本次实验开发的合约仅实现了基础的安全校验逻辑，在实际商业应用场景中，可添加 event 事件日志功能，对捐赠、拉黑地址、资金提取等所有关键操作进行记录，便于在区块链上对交易记录进行全程追溯，进一步提升合约的可审计性；还可添加捐赠金额校验规则，限制单次捐赠的最小和最大金额，规避恶意小额捐赠或异常大额捐赠带来的合约运行问题。

功能拓展：可基于本次实验的合约基础，增加捐赠者留言功能，让捐赠者在发起捐赠时添加

自定义备注信息；还可开发退款机制，当众筹目标在指定时间内未完成时，允许捐赠者撤回自身的捐赠资金，进一步丰富合约的实际业务应用场景。
区块链技术价值：去中心化捐赠合约的成功实现，证明了区块链技术能够有效解决传统中心化场景中存在的透明度低、信任缺失等核心问题。除捐赠场景外，该技术模式还可应用于众筹、慈善、公益等多个领域，通过“代码即规则”的方式，实现业务流程的公开、透明、可追溯。
开发规范重要性：智能合约的代码逻辑直接决定了资金的安全与否，一个微小的语法错误或逻辑漏洞，都可能导致合约出现安全问题，造成不可挽回的资金损失。因此在智能合约开发过程中，必须严格遵循开发规范，添加完善的条件校验逻辑，同时开展充分的测试工作，覆盖所有正常和异常的业务场景。

8.实验报告附件
代码：

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

contract TipJar {
    // 基础功能：管理员地址
    address public owner;
    // 进阶功能：黑名单映射
    mapping(address => bool) public blacklist;
    // 进阶功能：众筹目标 5 ETH（常量）
    uint public constant GOAL = 5 ether;
    // 进阶功能：数据分析-最高单笔捐赠、独立捐赠者数
    uint public highestDonation;
    uint public donorCount;
    mapping(address => bool) public hasDonated;
    // 进阶功能：时间锁-合约部署时间戳
    uint public deployTime;

    // 构造函数：部署时初始化管理员和部署时间
    constructor() {
        owner = msg.sender;
        deployTime = block.timestamp;
    }

    // 基础功能：捐赠接口（payable 可接收 ETH，整合所有进阶校验）
    function donate() public payable {
        require(!blacklist[msg.sender], "Address is blacklisted");
        require(address(this).balance + msg.value < GOAL, "Funding goal reached");
        require(msg.value > 0, "Donation amount must be greater than 0");

        // 更新最高单笔捐赠
        if (msg.value > highestDonation) {
```

```
            highestDonation = msg.value;
        }
        // 新捐赠者计数+1，避免重复计数
        if (!hasDonated[msg.sender]) {
            hasDonated[msg.sender] = true;
            donorCount++;
        }
    }

    // 基础功能：余额查询（view 只读，不消耗 Gas）
    function getBalance() public view returns (uint) {
        return address(this).balance;
    }

    // 基础功能：资金提取（整合管理员+时间锁校验）
    function withdraw() public {
        require(msg.sender == owner, "Only owner can withdraw");
        require(block.timestamp >= deployTime + 300, "Withdrawal not allowed yet: 5min cooling
period");
        require(address(this).balance > 0, "Contract has no ETH to withdraw");
        // 合约资金转账给管理员
        payable(owner).transfer(address(this).balance);
    }

    // 进阶功能：管理员添加黑名单（仅 owner 可操作）
    function addToBlacklist(address _addr) public {
        require(msg.sender == owner, "Only owner can manage blacklist");
        blacklist[_addr] = true;
    }

    // 进阶功能：管理员移除黑名单（可选，方便测试）
    function removeFromBlacklist(address _addr) public {
        require(msg.sender == owner, "Only owner can manage blacklist");
        blacklist[_addr] = false;
    }
}
```